

```
package com.escuela.api.controllers;
```

AlumnoController

```
@RestController
@RequestMapping("/alumnos")
@CrossOrigin(origins = "*")
```

```
public class AlumnoController {
```

```
    @Autowired
    private TareaProgramadaService tareaProgramadaService;
    @Autowired
    private AlumnoRepository alumnoRepository;
```

```
    // 1. LISTAR TODOS (Para el Profe)
    @GetMapping
    public List<Alumno> listar() {
        return alumnoRepository.findAll();
    }
```

```
    // 2. BUSCAR POR DNI (Para el Padre)
    @GetMapping("/buscar")
    public List<Alumno> buscarPorDni(@RequestParam String dni) {
        return alumnoRepository.findByDni(dni);
    }
```

```
    // 3. REGISTRO DE COMEDOR (Recuperando la elección del alumno)
    @PutMapping("/{id}/comedor")
    public ResponseEntity<?> actualizarComedor(@PathVariable Long id,
        @RequestParam boolean seQueda,
        @RequestParam(required = false) String turnoElegido) { // Agregamos esto
```

```
        // Mantenemos la restricción horaria (9:00 AM)
        LocalDateTime ahora = LocalDateTime.now();
        if (ahora.isAfter(LocalTime.of(9, 0))) {
            return ResponseEntity.status(HttpStatus.FORBIDDEN)
                .body("El registro para el comedor cierra a las 9:00 AM.");
        }
```

```
        Alumno alumno = alumnoRepository.findById(id).orElse(null);
        if (alumno == null) {
            return ResponseEntity.notFound().build();
        }
```

```
        if (seQueda) {
            // RESTRICCIÓN DE LA 275: Si tiene taller a las 13:00, no puede elegir 12:40
            if ("13:00".equals(alumno.getHoraInicioTaller()) && "12:40".equals(turnoElegido)) {
                return ResponseEntity.status(HttpStatus.BAD_REQUEST)

```

```

        .body("No podés elegir 12:40 porque tenés taller a las 13:00 hs.");
    }

    // Guardamos el turno que el alumno eligió en el Front
    alumno.setTurnoComedor(turnoElegido);
} else {
    alumno.setTurnoComedor("No");
}

return ResponseEntity.ok(alumnoRepository.save(alumno));
}

// 4. NUEVO: ACTUALIZAR NOTA (ABM Profe)
@PutMapping("/{id}/nota")
public ResponseEntity<Alumno> actualizarNota(@PathVariable Long id, @RequestParam Double
valor) {
    return alumnoRepository.findById(id).map(a -> {
        a.setNota(valor);
        return ResponseEntity.ok(alumnoRepository.save(a));
    }).orElse(ResponseEntity.notFound().build());
}

// 5. NUEVO: ROTACIÓN DE TALLER (ABM Profe)
@PutMapping("/{id}/taller")
public ResponseEntity<Alumno> actualizarTaller(@PathVariable Long id, @RequestParam String
nuevoHorario) {
    return alumnoRepository.findById(id).map(a -> {
        a.setHoraInicioTaller(nuevoHorario);
        return ResponseEntity.ok(alumnoRepository.save(a));
    }).orElse(ResponseEntity.notFound().build());
}

@GetMapping("/comedor/resumen")
public Map<String, Long> getResumenComedor() {
    List<Object[]> resultados = alumnoRepository.obtenerConteosComedor();
    Map<String, Long> resumen = new HashMap<>();
    for (Object[] fila : resultados) {
        resumen.put((String) fila[0], (Long) fila[1]);
    }
    return resumen;
}

@PutMapping("/reset-comedor")
public ResponseEntity<String> resetearComedor() {
    tareaProgramadaService.resetDiarioComedor(); // Llama a la función que ya tenés
    return ResponseEntity.ok("Comedor reseteado correctamente");
}

```

```
}
```

```
package com.escuela.api.controllers;
```

CalentitoController

```
@CrossOrigin(origins = "*", allowedHeaders = "*", methods = {RequestMethod.GET,
RequestMethod.POST, RequestMethod.PUT, RequestMethod.OPTIONS})
```

```
@RestController
```

```
@RequestMapping("/api/calentitos")
```

```
public class CalentitoController {
```

```
    @Autowired
```

```
    private PedidoRepository pedidoRepository;
```

```
    @Autowired
```

```
    private AlumnoRepository alumnoRepository;
```

```
    // Función que valida las ventanas de recreo
```

```
    public String determinarRecreo() {
```

```
        LocalDateTime ahora = LocalDateTime.now();
```

```
        // --- TURNO MAÑANA ---
```

```
        // Ventana 1: 6:00 a 8:00
```

```
        if (ahora.isAfter(LocalTime.of(6, 0)) && ahora.isBefore(LocalTime.of(8, 0))) {
```

```
            return "PRIMER_RECREO_MAÑANA";
```

```
        }
```

```
        // Ventana 2: 8:40 a 9:30
```

```
        if (ahora.isAfter(LocalTime.of(8, 40)) && ahora.isBefore(LocalTime.of(9, 30))) {
```

```
            return "SEGUNDO_RECREO_MAÑANA";
```

```
        }
```

```
        // --- TURNO TARDE ---
```

```
        // Ventana 3: 12:00 a 13:00
```

```
        if (ahora.isAfter(LocalTime.of(12, 0)) && ahora.isBefore(LocalTime.of(13, 15))) {
```

```
            return "PRIMER_RECREO_TARDE";
```

```
        }
```

```
        // Ventana 4: 13:45 a 14:45
```

```
        if (ahora.isAfter(LocalTime.of(13, 45)) && ahora.isBefore(LocalTime.of(14, 45))) {
```

```
            return "SEGUNDO_RECREO_TARDE";
```

```
        }
```

```
        return "CERRADO";
```

```
    }
```

```
    @PostMapping("/pedir")
```

```
    public ResponseEntity<String> registrarPedido(@RequestBody PedidoDTO datos) {
```

```
        String recreo = determinarRecreo();
```

```

if (recreo.equals("CERRADO")) {
    return ResponseEntity.ok("HORARIO_CERRADO");
}

try {
    Pedido nuevoPedido = new Pedido();
    Alumno alumno = alumnoRepository.findById(datos.getAlumnoId()).get();
    nuevoPedido.setAlumno(alumno);
    nuevoPedido.setMetodoPago(datos.getMetodoPago());
    nuevoPedido.setRecreo(recreo);
    nuevoPedido.setFecha(LocalDate.now());
    nuevoPedido.setEntregado(false);

    nuevoPedido.setPagado(!datos.getMetodoPago().equalsIgnoreCase("EFECTIVO"));

    pedidoRepository.save(nuevoPedido);

    return ResponseEntity.ok("iPedido anotado para el " + recreo + "!");
} catch (Exception e) {
    return ResponseEntity.internalServerError().body("Error al guardar: " + e.getMessage());
}

@GetMapping("/pendientes")
public List<Pedido> obtenerPendientes() {
    return pedidoRepository.findByEntregadoFalse();
}

@PutMapping("/entregar/{id}")
public ResponseEntity<String> marcarEntregado(@PathVariable Long id) {
    return pedidoRepository.findById(id).map(pedido -> {
        pedido.setEntregado(true);
        pedidoRepository.save(pedido);
        return ResponseEntity.ok("iTostado entregado con éxito!");
    }).orElse(ResponseEntity.notFound().build());
}
}

```

```
package com.escuela.api.controllers;
```

```
PedidoDTO
```

```

public class PedidoDTO {

    private Long alumnoId;
    private String metodoPago;

```

```

public PedidoDTO(Long alumnoId, String metodoPago) {
    this.alumnoId = alumnoId;
    this.metodoPago = metodoPago;
}

public Long getAlumnoId() {
    return alumnoId;
}

public void setAlumnoId(Long alumnoId) {
    this.alumnoId = alumnoId;
}

public String getMetodoPago() {
    return metodoPago;
}

public void setMetodoPago(String metodoPago) {
    this.metodoPago = metodoPago;
}
}

```

```

package com.escuela.api.controllers;

```

BotController

```

@RestController
@RequestMapping("/bot")
@CrossOrigin(origins = "http://127.0.0.1:5500")
public class BotController {

    @Autowired
    private AlumnoRepository repository;

    @GetMapping("/preguntar")
    public String responder(@RequestParam String msg) {
        String pregunta = msg.toLowerCase();

        // Lógica para el comedor
        if (pregunta.contains("comen") || pregunta.contains("cuantos")) {
            // Filtramos directamente en la lista los que NO tienen "No"
            long cantidad = repository.findAll().stream()
                .filter(a -> a.getTurnoComedor() != null &&
                    !a.getTurnoComedor().equalsIgnoreCase("No"))
                .count();

```

```

        return "Hoy se quedan a comer " + cantidad + " alumnos. ¿Querés que te pase la lista?";
    }

    // Saludo inicial que vimos en tu captura
    if (pregunta.contains("hola")) {
        return "¡Hola Román! Soy el asistente de la Escuela. ¿En qué puedo ayudarte con los
alumnos?";
    }

    if(pregunta.contains("quien te creó?")){
        return "Alumnos de 6to año de la terminalidad Informática";
    }

    return "Todavía no entiendo esa pregunta, pero puedo decirte cuántos alumnos comen hoy.";
}
}

```

```

package com.escuela.api.models;

```

Alumno.java

```

@Entity
@Table(name = "alumnos")

public class Alumno {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nombre;
    private String apellido;
    private String dni;
    private boolean pideCalentito;
    @Column(name = "telefono_padre")
    private String telefonoPadre;
    @Column(name = "hora_inicio_taller")
    private String horaInicioTaller;
    @Column(name = "turno_comedor")
    private String turnoComedor;
    private Double nota;

    public Double getNota() {
        return nota;
    }

    public void setNota(Double nota) {
        this.nota = nota;
    }
}

```

```
public Alumno() {  
}  
  
public Long getId() {  
    return id;  
}  
  
public void setId(Long id) {  
    this.id = id;  
}  
  
public String getNombre() {  
    return nombre;  
}  
  
public void setNombre(String nombre) {  
    this.nombre = nombre;  
}  
  
public String getApellido() {  
    return apellido;  
}  
  
public void setApellido(String apellido) {  
    this.apellido = apellido;  
}  
  
public String getDni() {  
    return dni;  
}  
  
public void setDni(String dni) {  
    this.dni = dni;  
}  
  
public String getTelefonoPadre() {  
    return telefonoPadre;  
}  
  
public void setTelefonoPadre(String telefonoPadre) {  
    this.telefonoPadre = telefonoPadre;  
}  
  
public String getHoraInicioTaller() {  
    return horaInicioTaller;  
}  
}
```

```

public void setHoraInicioTaller(String horaInicioTaller) {
    this.horaInicioTaller = horaInicioTaller;
}

public String getTurnoComedor() {
    return turnoComedor;
}

public void setTurnoComedor(String turnoComedor) {
    this.turnoComedor = turnoComedor;
}
}

```

```
package com.escuela.api.repositories;
```

AlumnoRepository

```

@Repository
public interface AlumnoRepository extends JpaRepository<Alumno, Long> {

    // 1. Buscar coincidencia exacta de DNI
    List<Alumno> findByDni(String dni);

    // 2. Buscar por nombre
    List<Alumno> findByNombreContainingIgnoreCase(String nombre);

    // 3. Obtener conteos para el resumen del panel
    @Query("SELECT a.turnoComedor, COUNT(a) FROM Alumno a WHERE a.turnoComedor != 'No' AND a.turnoComedor IS NOT NULL GROUP BY a.turnoComedor")
    List<Object[]> obtenerConteosComedor();

    // 4. RESET MASIVO PROFESIONAL
    @Transactional
    @Modifying(clearAutomatically = true)
    @Query("UPDATE Alumno a SET a.turnoComedor = 'No' WHERE a.id > 0");
    void resetearComedorMasivo();
}

```

```

@Repository
public interface PedidoRepository extends JpaRepository<Pedido, Long> {
    // Este método les va a servir para ver solo los pedidos de hoy
    List<Pedido> findByFechaAndEntregadoFalse(LocalDate fecha);
    // Esto le dice a Spring: "Buscame en la BD todos los que tengan entregado = false"
    List<Pedido> findByEntregadoFalse();
}

```



```
package com.escuela.api.models;
```

Pedido.java

```
@Entity
```

```
@Table(name = "pedidos_calentitos")
```

```
public class Pedido {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    // Usamos el objeto Alumno para que Hibernate traiga nombre y apellido automáticamente.
```

```
    @ManyToOne
```

```
    @JoinColumn(name = "alumno_id")
```

```
    private Alumno alumno;
```

```
    private String recreo;    // PRIMER_RECREO o SEGUNDO_RECREO
```

```
    private String metodoPago; // EFECTIVO, TRANSFERENCIA, QR
```

```
    private boolean pagado;    // True si es Transf/QR
```

```
    private boolean entregado; // Para que los de 6to marquen cuando lo dan
```

```
    private LocalDate fecha;   // Para filtrar los pedidos del día
```

```
    public Pedido() {
```

```
        this.entregado = false; // Por defecto no está entregado
```

```
        this.fecha = LocalDate.now(); // Se setea la fecha actual al crear el pedido
```

```
    }
```

```
    public Pedido(Long id, Alumno alumno, String recreo, String metodoPago, boolean pagado, boolean entregado, LocalDate fecha) {
```

```
        this.id = id;
```

```
        this.alumno = alumno;
```

```
        this.recreo = recreo;
```

```
        this.metodoPago = metodoPago;
```

```
        this.pagado = pagado;
```

```
        this.entregado = entregado;
```

```
        this.fecha = fecha;
```

```
    }
```

```
    public Long getId() {
```

```
        return id;
```

```
    }
```

```
    public void setId(Long id) {
```

```
        this.id = id;
```

```
    }
```

```
    public Alumno getAlumno() {
```

```

        return alumno;
    }

    public void setAlumno(Alumno alumno) {
        this.alumno = alumno;
    }

    public String getRecreo() {
        return recreo;
    }

    public void setRecreo(String recreo) {
        this.recreo = recreo;
    }

    public String getMetodoPago() {
        return metodoPago;
    }

    public void setMetodoPago(String metodoPago) {
        this.metodoPago = metodoPago;
    }

    public boolean isPagado() {
        return pagado;
    }

    public void setPagado(boolean pagado) {
        this.pagado = pagado;
    }

    public boolean isEntregado() {
        return entregado;
    }

    public void setEntregado(boolean entregado) {
        this.entregado = entregado;
    }

    public LocalDate getFecha() {
        return fecha;
    }

    public void setFecha(LocalDate fecha) {
        this.fecha = fecha;
    }
}

```